6 . Bidirectional recurrent neural network (RNN) to predict sequences in time-series data.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import time

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_erro
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN, Bidirectiona

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/

df = pd.read_csv(url)

data = df["Passengers"].values.reshape(-1, 1)

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)

look_back = 12

X = []
y = []

for i in range(len(scaled_data) - look_back):
    X.append(scaled_data[i:i + look_back])
    y.append(scaled_data[i + look_back])

X = np.array(X)
y = np.array(y)

train_size = int(len(X) * 0.8)

X_train = X[:train_size]
X_test = X[train_size:]
y_train = y[:train_size]
y_test = y[train_size:]

X_train_rnn = X_train
X_test_rnn = X_test
```

```
X_train_fnn = X_train.reshape(X_train.shape[0], X_train.shape[1])
X_test_fnn = X_test.reshape(X_test.shape[0], X_test.shape[1])

rnn_model = Sequential([
    Bidirectional(SimpleRNN(64, activation="tanh"), input_shape=(l
    Dense(32, activation="relu"),
    Dense(1)
])

rnn_model.compile(
    optimizer="adam",
    loss="mse"
)

start = time.time()

rnn_history = rnn_model.fit(
    X_train_rnn,
    y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.1,
    verbose=1
)

rnn_time = time.time() - start

fnn_model = Sequential([
    Dense(64, activation="relu", input_shape=(look_back,)),
    Dense(32, activation="relu"),
    Dense(1)
])

fnn_model.compile(
    optimizer="adam",
    loss="mse"
)

start = time.time()

fnn_history = fnn_model.fit(
    X_train_fnn,
    y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.1,
    verbose=1
)
```

```
fnn_time = time.time() - start

rnn_pred = rnn_model.predict(X_test_rnn)
fnn_pred = fnn_model.predict(X_test_fnn)

rnn_pred = scaler.inverse_transform(rnn_pred)
fnn_pred = scaler.inverse_transform(fnn_pred)
y_actual = scaler.inverse_transform(y_test)

rnn_mae = mean_absolute_error(y_actual, rnn_pred)
rnn_rmse = np.sqrt(mean_squared_error(y_actual, rnn_pred))

fnn_mae = mean_absolute_error(y_actual, fnn_pred)
fnn_rmse = np.sqrt(mean_squared_error(y_actual, fnn_pred))

print("\nBiRNN Performance")
print("MAE :", rnn_mae)
print("RMSE:", rnn_rmse)
print("Training Time:", rnn_time, "seconds")

print("\nFeed-Forward Neural Network Performance")
print("MAE :", fnn_mae)
print("RMSE:", fnn_rmse)
print("Training Time:", fnn_time, "seconds")

results = pd.DataFrame({
    "Model": ["Bidirectional RNN", "Feed-Forward NN"],
    "MAE": [rnn_mae, fnn_mae],
    "RMSE": [rnn_rmse, fnn_rmse],
    "Training Time": [rnn_time, fnn_time]
})

print("\nComparison")
print(results)

plt.figure(figsize=(12, 5))
plt.plot(y_actual, label="Actual")
plt.plot(rnn_pred, label="BiRNN Prediction")
plt.plot(fnn_pred, label="FNN Prediction")
plt.title("Actual vs Predicted Airline Passengers")
plt.xlabel("Time")
plt.ylabel("Passengers")
plt.legend()
plt.show()

plt.figure(figsize=(10, 5))
plt.plot(rnn_history.history["loss"], label="BiRNN Training Loss")
plt.plot(rnn_history.history["val_loss"], label="BiRNN Validation
plt.title("BiRNN Training and Validation Loss")
```

```
plt.title("BiRNN Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

plt.figure(figsize=(10, 5))
plt.plot(fnn_history.history["loss"], label="FNN Training Loss")
plt.plot(fnn_history.history["val_loss"], label="FNN Validation Los
plt.title("FNN Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

plt.figure(figsize=(8, 5))
plt.bar(
    ["BiRNN", "Feed-Forward NN"],
    [rnn_mae, fnn_mae]
)
plt.title("MAE Comparison")
plt.ylabel("MAE")
plt.show()

plt.figure(figsize=(8, 5))
plt.bar(
    ["BiRNN", "Feed-Forward NN"],
    [rnn_rmse, fnn_rmse]
)
plt.title("RMSE Comparison")
plt.ylabel("RMSE")
plt.show()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/bidir
  super().__init__(**kwargs)
Epoch 1/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 6s 84ms/step - loss: 0.0210 - val_loss: 0
Epoch 2/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0066 - val_loss: 0
Epoch 3/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0040 - val_loss: 0
Epoch 4/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0027 - val_loss: 0
```